Quantum Models for Corner Detection & Keypoint Extraction

A Quantum Local Feature Classification Task for SLAM / AR / Robotics: Task Definition, Classical Baselines, and Quantum Algorithm Literature Review

NJUquantum

2026-06-24

Contents

1 Project Positioning

This topic is essentially an attempt to reformulate **local feature point detection** in classical computer vision as a small-scale QML / QNN task. It should not be interpreted as "using quantum computing to replace the whole vision system." A more realistic interpretation is:

> Given a local image patch, determine whether the center point is a corner, junction, or keypoint, while trying to remain stable under low circuit depth, few parameters, small datasets, and strong noise.

Therefore, the goal is not to feed an entire image directly into a quantum model. The goal is to design a low-resource, patch-level, ablation-friendly **quantum local feature classifier**:

$$P_i \mapsto y_i \in \{0, 1\},$$

where P_i is a local patch centered at pixel i, and $y_i = 1$ means that the point is a corner, junction, or stable keypoint.

A clearer project title is:

Quantum Local Feature Classifier for Robust Corner and Keypoint Detection

A more formal title is:

Low-depth Quantum Local Feature Classification for Noise-robust Corner and Keypoint Detection

2 What Are Corners and Keypoints?

In an image, an **edge** is a location where intensity changes sharply along one dominant direction, such as an object boundary. A **corner** is a location where intensity changes strongly along two or more directions, such as a table corner, a checkerboard intersection, or a building window corner.

A useful local interpretation is:

- **Flat region**: moving in any direction causes little intensity change.
- **Edge region**: moving perpendicular to the edge causes a large change, while moving along the edge causes a small change.
- **Corner region**: moving in multiple directions causes large changes.

Corners are therefore useful for visual localization, matching, and tracking. Compared with ordinary edge points, corners are usually easier to rediscover across frames, viewpoints, and scales.

A **keypoint** is a broader concept. A keypoint usually contains:

- position: (x, y);
- scale;
- orientation;
- response score;
- descriptor, such as BRIEF, ORB, or SIFT.

A corner is a common source of keypoints, but not all keypoints are corners. Keypoints may also be blobs, junctions, strong texture points, or other stable local structures.

3 What Is a Junction?

A **junction** is a location where multiple edges, line segments, or contours meet, intersect, merge, or split. Common examples include:

- T-junction;
- Y-junction;
- X-junction;
- L-corner;
- checkerboard intersection;
- road intersection;
- structural intersection in buildings.

Junctions carry more structural meaning than ordinary corners. In robotics, SLAM, and AR, junctions often correspond to stable geometric structures and may be more suitable for long-term tracking and map construction than simple edge points.

4 Why Visual SLAM, AR/VR, and Robotics Need These Points

Visual SLAM means **simultaneous localization and mapping**. It estimates camera motion from consecutive image frames while constructing a map of the environment.

A typical visual SLAM pipeline includes:

1. feature extraction;
2. feature matching / tracking;
3. visual odometry;
4. local mapping;
5. loop closure;
6. global optimization / bundle adjustment.

Feature extraction is usually the first step of the visual front end. If keypoints are unstable, matching fails. If too many keypoints are detected, computation becomes expensive. If keypoints are poorly distributed, pose estimation becomes unstable.

Thus, the real objective of this project is not to produce a visually pleasing corner map, but rather:

> to find local keypoints that are useful for tracking and matching under noise, low light, low resolution, motion blur, and viewpoint changes.

This also means that evaluation should not only use pixel-wise accuracy. More relevant metrics include repeatability, localization error, matching score, and impact on downstream pose estimation.

5 Harris Corner Detector

5.1 Basic Idea

Harris is a classical corner detection algorithm. Its core idea is that a corner is a local region with large intensity changes in all directions. The original paper by Harris and Stephens unified corner and edge detection through a local autocorrelation function and a local gradient matrix, and emphasized its use in feature tracking for image sequences.[1]

First compute image gradients:

$$I_x = \frac{\partial I}{\partial x}, \quad I_y = \frac{\partial I}{\partial y}.$$

Then construct the **structure tensor**, also called the **second-moment matrix**, over a local window:

[1] C. Harris and M. Stephens, "A Combined Corner and Edge Detector," Proceedings of the 4th Alvey Vision Conference, 1988. https://www.bmva-archive.org.uk/bmvc/1988/avc-88-023.pdf

$$M = \sum_{(u,v)\in W} w(u,v) \begin{pmatrix} I_x^2 & I_x I_y \\ I_x I_y & I_y^2 \end{pmatrix}.$$

Intuitively, M records the distribution of local gradient directions.

The Harris response is usually written as:

$$R = \det(M) - k\,\mathrm{tr}(M)^2.$$

where

$$\det(M) = \lambda_1 \lambda_2, \quad \mathrm{tr}(M) = \lambda_1 + \lambda_2,$$

and λ_1, λ_2 are the strengths of intensity change along two principal local directions.

The decision rule is:

- both λ_1, λ_2 small: flat region;
- one large and one small: edge;
- both large: corner.

Therefore, Harris is a natural classical baseline for QML-based corner detection.

5.2 Implications for This Project

The strength of Harris is that it does not rely directly on raw pixels, but on local gradient statistics. This is important for a QML project: instead of feeding a full image or a large raw patch into a quantum circuit, it is more practical to first construct low-dimensional and physically meaningful features such as:

$$[I_x, I_y, I_x^2, I_y^2, I_x I_y, \lambda_1, \lambda_2, R].$$

This greatly reduces encoding cost and is better suited to small NISQ-stage QNNs.

6 FAST: Features from Accelerated Segment Test

6.1 Basic Idea

FAST stands for **Features from Accelerated Segment Test** and emphasizes speed. Rosten and Drummond introduced a high-speed corner detector using machine learning in ECCV 2006, reporting significant speed advantages over Harris and SUSAN. They also noted that because FAST examines as few pixels as possible for speed, it can be less noise-robust than smoother detectors.[2]

[2] E. Rosten and T. Drummond, "Machine Learning for High-Speed Corner Detection," ECCV 2006. https://www.edwardrosten.com/work/rosten_2006_machine.pdf

FAST works as follows:

1. choose a center pixel p;
2. take the 16 pixels on a Bresenham circle around it;
3. if a contiguous arc of pixels is all much brighter or much darker than the center, then p is considered a corner.

Given threshold t, if a contiguous segment of circle pixels satisfies:

$$I_i > I_p + t$$

or

$$I_i < I_p - t,$$

then the center point p is a candidate corner.

FAST is fast and therefore suitable for real-time video, tracking, SLAM, and embedded vision. Its limitations are also clear: the original FAST lacks orientation, is sensitive to scale and image domain changes, and may be less stable under noise.

6.2 Implications for This Project

FAST is a strong real-time baseline. If a QNN model is more robust than Harris but much slower than FAST, its application value is limited. Therefore this project should report:

- accuracy / F1;
- repeatability;
- inference cost;
- circuit depth;
- number of two-qubit gates;
- comparison against FAST in both speed and stability.

7 ORB: Oriented FAST and Rotated BRIEF

7.1 Basic Idea

ORB means **Oriented FAST and Rotated BRIEF**. Rublee et al. proposed ORB at ICCV 2011 as a fast alternative to SIFT and SURF. ORB combines the FAST detector with the BRIEF descriptor, and improves rotation robustness through orientation assignment and rotated BRIEF. The paper reports that ORB can achieve performance close to SIFT in many scenarios while being around two orders of magnitude faster.[3]

ORB typically includes:

[3] E. Rublee, V. Rabaud, K. Konolige, and G. Bradski, "ORB: An efficient alternative to SIFT or SURF," ICCV 2011. https://ieeexplore.ieee.org/document/6126544

- FAST for rapid candidate keypoint detection;
- Harris score for selecting strong keypoints;
- orientation assignment;
- BRIEF descriptor generation;
- image pyramid for multi-scale features.

ORB is important because it does not only detect keypoints but also provides descriptors for matching. Therefore it is closer to real SLAM / AR / robotics pipelines than Harris or FAST alone.

7.2 Implications for This Project

If the QNN only outputs a corner probability, it corresponds to a detector, not a complete ORB pipeline. A practical short-term project should first focus on the detector:

$$f_\theta(P_i) \to p_{\text{corner}}.$$

A later extension could learn descriptors:

$$g_\theta(P_i) \to d_i,$$

where d_i is a local descriptor used for matching.

A stable research path is:

1. Stage 1: QNN for corner/keypoint classification;
2. Stage 2: classical BRIEF / ORB descriptor for matching;
3. Stage 3: quantum or hybrid descriptor learning.

8 Image Gradients and Hessian-like Structures

8.1 Image Gradient

The image gradient is the first derivative of image intensity:

$$\nabla I = (I_x, I_y).$$

It tells us the direction of fastest intensity change. Edge detection often uses gradient magnitude:

$$\|\nabla I\| = \sqrt{I_x^2 + I_y^2}.$$

8.2 Hessian

The Hessian is the matrix of second-order derivatives:

$$H = \begin{pmatrix} I_{xx} & I_{xy} \\ I_{xy} & I_{yy} \end{pmatrix}.$$

It describes local curvature. Hessian-like structures are widely used for blob, ridge, junction, and corner detection. For example, the determinant of the Hessian captures local two-dimensional intensity changes.

8.3 Implications for QNN Input

In this project, "mapping image gradients or Hessian-like structures into a QNN" means:

> do not feed the full image into the quantum circuit; instead, first extract classical local visual features from each patch and then encode those low-dimensional features into the quantum circuit.

Possible input features include:

$$[I_x, I_y, I_{xx}, I_{xy}, I_{yy}, \lambda_1, \lambda_2, R, \det(H), \mathrm{tr}(H)].$$

These features have clear geometric meaning and reduce quantum encoding complexity.

9 What Exactly Does QNN / QML Do Here?

9.1 Patch-level Classification

Given a local $k \times k$ image patch, such as 5×5 or 7×7, the model determines whether the center pixel is a corner or keypoint:

$$f_\theta(P) \to p_{\text{corner}}.$$

Here P is a local patch and f_θ is a QNN.

Possible inputs include:

1. raw patch pixels;
2. gradient features;
3. structure tensor features;
4. Hessian features;
5. Harris / FAST score;
6. multi-scale patch features;
7. local plus context features.

9.2 Quantum Encoding Methods

Possible QNN encodings include:

- angle encoding;
- amplitude encoding;
- basis encoding;
- data re-uploading circuits;
- quantum convolutional circuits;
- small variational classifiers.

Under NISQ constraints, angle encoding or data re-uploading is recommended because they are easier to implement, debug, and ablate.

For example, angle encoding can map a feature x_i into a rotation:

$$|0\rangle \xrightarrow{R_y(x_1)} \xrightarrow{R_z(x_2)} \cdots$$

The output can be obtained by measuring a Pauli-Z expectation value:

$$p_{\text{corner}} = \sigma\left(\langle Z_0 \rangle\right).$$

10 Roles of Entanglement and Superposition

It is important not to overstate quantum effects.

10.1 Superposition

Superposition means that a quantum state can carry amplitudes over multiple computational basis states. In image tasks it is sometimes described as representing many pixels or features in parallel. However, the bottleneck is data encoding: loading classical image data into a quantum state itself has a cost.

Therefore, this project should not simply claim that "superposition accelerates image processing." Instead, it should analyze:

- encoding cost;
- measurement cost;
- number of qubits;
- circuit depth;
- fair comparison with classical models using the same input features.

10.2 Entanglement

Entanglement can create non-classical correlations among features encoded on different qubits. For a local image patch, this may correspond to interactions between:

- horizontal and vertical gradients;
- multi-scale features;
- different edge directions;

- local features and contextual features.

However, a short-term paper should not claim that entanglement brings quantum advantage. A more testable question is:

> Under the same parameter count and circuit depth, does an entangling QNN become more robust than a product-state circuit without entanglement?

Recommended ablations:

Model	Purpose
QNN without entanglement	Test whether single-qubit nonlinearities are enough
Ring-entangled QNN	Test local feature coupling
Fully-entangled QNN	Test whether strong coupling helps
Classical MLP with same features	Test whether the gain comes from ordinary nonlinearity
Harris / FAST / ORB	Compare against strong classical vision baselines

11 Local-global Feature Encoding

A corner is local, but whether a keypoint is useful often depends on broader context.

For example:

- a corner in a repetitive texture may be unstable for matching;
- a junction in a low-texture region may be useful;
- SLAM needs keypoints to be distributed across the image;
- strong keypoints clustered in one small region may still be bad for localization.

Thus local-global feature encoding means:

- **local**: gradient, Hessian, Harris / FAST response of the center patch;
- **global / context**: image location, scale, local texture statistics, local keypoint density, and image-pyramid statistics.

A short-term simplified version is:

$$x = [x_{\text{local}}, x_{\text{context}}].$$

where:

- x_{local} is a feature vector from a 5×5 patch;
- x_{context} is a statistic from a 15×15 context patch or image pyramid.

A QNN may attempt to entangle local qubits with context qubits to test whether local-context coupling improves robustness.

12 Quantum Edge AI / Resource-constrained Devices

Edge AI means running AI on edge devices such as:

- phones;
- AR/VR glasses;
- robots;
- drones;
- IoT cameras;
- vehicle devices.

Resource-constrained devices usually face:

- low compute power;
- small memory;
- limited battery;
- strict latency requirements;
- unstable network connections.

"Quantum edge AI" is currently more of a future vision. A short-term project should not claim real deployment. A more realistic wording is:

> low-depth, few-qubit, parameter-efficient quantum local feature extractor.

Therefore, evaluation should include:

- number of qubits;
- circuit depth;
- number of two-qubit gates;
- number of trainable parameters;
- inference cost;
- number of shots;
- noise robustness;
- fair comparison with lightweight classical baselines.

13 Dataset and Label Construction

13.1 Label Sources

Labels may come from:

1. high-response Harris points;
2. FAST / ORB keypoints;
3. synthetic ground-truth corners;
4. checkerboard or geometric shapes;
5. repeatably tracked keypoints in SLAM datasets;
6. repeatable keypoints in multi-view image pairs.

The most stable first version is:

synthetic ground truth + Harris / FAST pseudo-labels + real-image validation.

13.2 Recommended Data Design

First use synthetic images:

- L-corners;
- T-junctions;
- X-junctions;
- Y-junctions;
- checkerboards;
- random polygons;
- line intersections;
- noisy geometric shapes.

Generate ground-truth corner / junction coordinates, and then sample positive and negative patches.

Then add real images:

- checkerboard calibration images;
- HPatches;
- EuRoC / TUM-VI / KITTI frames;
- indoor low-light, blurred, and noisy images.

14 Baseline Design

Classical baselines are mandatory. Without them, the paper will not be convincing.

Baseline	Role
Sobel / Canny	Edge-level baseline
Harris	Corner detector baseline
FAST	Real-time keypoint baseline
ORB	Practical keypoint + descriptor pipeline
Classical MLP	Check whether QNN only benefits from good features
Small CNN	Check against lightweight deep learning
QNN without entanglement	Ablate entanglement
QNN with entanglement	Main model
Noise-aware QNN	Test robustness

For fairness, the classical MLP should use exactly the same input features as the QNN:

$$x = [I_x, I_y, I_{xx}, I_{xy}, I_{yy}, \lambda_1, \lambda_2, R].$$

Otherwise, it is impossible to tell whether performance comes from the quantum circuit or from feature engineering.

15 Evaluation Metrics

If the task is only pixel-wise classification, use:

- precision;
- recall;
- F1;
- ROC-AUC;
- PR-AUC.

For keypoint detection, more relevant metrics are:

- repeatability;
- localization error;
- matching score;
- number of detected keypoints;
- distribution uniformity;
- runtime / inference cost.

For SLAM-oriented evaluation, also consider:

- tracking success rate;
- pose estimation error;
- number of inlier matches;
- RANSAC inlier ratio;
- trajectory error after bundle adjustment.

The first version should focus on:

1. F1 / PR-AUC;
2. repeatability;
3. localization error;
4. noise robustness;
5. circuit resource cost.

Do not start by integrating a full SLAM system, because that would be too engineering-heavy.

16 Recommended Model Route

16.1 Input Features

For each patch, extract:

$$x = [I_x, I_y, I_{xx}, I_{xy}, I_{yy}, I_x^2, I_y^2, I_x I_y, \lambda_1, \lambda_2, R, \det(H), \mathrm{tr}(H)].$$

Depending on the number of qubits, reduce this to 4, 6, 8, or 10 features.

16.2 QNN Structure

A simple feasible circuit:

1. angle encoding layer;
2. parameterized single-qubit rotations;
3. ring entanglement layer;
4. data re-uploading;
5. final measurement.

For example:

$$U(x,\theta) = U_L(x,\theta_L)\cdots U_2(x,\theta_2)U_1(x,\theta_1).$$

Output:

$$\hat{y} = \sigma\left(w\langle Z_0\rangle + b\right).$$

16.3 Loss Function

For binary classification, use binary cross entropy:

$$\mathcal{L} = -\frac{1}{N}\sum_i \left[y_i \log \hat{y}_i + (1-y_i)\log(1-\hat{y}_i)\right].$$

Because corners are usually much rarer than non-corners, class weighting may be needed.

17 Noise Robustness Experiments

Add the following perturbations:

Perturbation	Purpose
Gaussian noise	Simulate sensor noise
Salt-and-pepper noise	Simulate dead pixels and impulse noise
Motion blur	Simulate camera motion
Low light	Simulate poor illumination
Contrast change	Simulate exposure change
Downsampling	Simulate low resolution
Rotation / scale	Check geometric stability

The central question is:

> Under small data, low parameter count, and noisy input, is the QNN more stable than a classical MLP or Harris / FAST?

If it does not beat strong baselines, the contribution can still be framed as:

> a systematic benchmark of the capability boundary of low-depth QNNs for corner/keypoint extraction, including ablations on circuit structure, input features, and noise robustness.

18 Classical Vision and SLAM Literature Review

18.1 Harris and Stephens, "A Combined Corner and Edge Detector"

18.1.1 Content

Harris and Stephens proposed the classic Harris corner detector. It analyzes local image autocorrelation under small shifts and leads naturally to the second-moment matrix / structure tensor. The method distinguishes flat regions, edges, and corners.

18.1.2 Core Contributions

1. Formulated corner detection as a local image autocorrelation problem;
2. constructed a second-moment matrix from local gradient statistics;
3. used matrix eigenvalues to classify local structure;
4. proposed the Harris response

$$R = \det(M) - k\,\mathrm{tr}(M)^2.$$

18.1.3 Relevance to This Project

Harris is the most important classical baseline for this project. It also inspires QNN input design: instead of raw pixels, use structure tensor, eigenvalues, and Harris response as low-dimensional geometric features.

18.1.4 Limitations

Harris is not fully scale-robust and is usually combined with an image pyramid or multi-scale strategy. It is also not a descriptor, so it cannot complete matching by itself.

18.2 Rosten and Drummond, "Machine Learning for High-Speed Corner Detection" / FAST

18.2.1 Content

FAST proposes a high-speed corner detector. It compares the center pixel with 16 pixels on a circle and quickly decides whether the center point is a corner. The paper uses machine learning to optimize the decision order.

18.2.2 Core Contributions

1. Converts corner detection into a segment test;
2. uses 16 pixels on a Bresenham circle;
3. introduces a learned decision tree for speed;

4. targets real-time tracking, SLAM, and matching.

18.2.3 Relevance to This Project

FAST is an important speed baseline. If a QNN is not close to FAST in efficiency, it must clearly demonstrate a different benefit, such as noise robustness, small-data performance, or low-parameter representation.

18.2.4 Limitations

The original FAST lacks orientation and scale information, and is sensitive to noise, scale changes, and domain shift. It is therefore often used as part of ORB.

18.3 Rublee et al., "ORB: An Efficient Alternative to SIFT or SURF"

18.3.1 Content

ORB combines the FAST detector with the BRIEF descriptor and improves rotation robustness through orientation assignment and rotated BRIEF. It aims to be a fast alternative to SIFT / SURF.

18.3.2 Core Contributions

1. Uses FAST for candidate keypoints;
2. uses Harris measure to rank keypoints;
3. assigns orientation;
4. uses a rotation-aware BRIEF descriptor;
5. balances speed and matching performance.

18.3.3 Relevance to This Project

ORB is the practical engineering baseline. Comparing only with Harris or FAST would make the task too narrow. ORB clarifies how a QNN detector relates to a real keypoint pipeline.

18.3.4 Limitations

ORB is still a hand-crafted feature method. It can fail under low texture, strong blur, severe illumination changes, or repetitive texture.

18.4 Mur-Artal et al., "ORB-SLAM: A Versatile and Accurate Monocular SLAM System"

18.4.1 Content

ORB-SLAM is a classic feature-based monocular SLAM system. It uses ORB features for tracking, mapping, relocalization, and loop closing.[4]

[4] R. Mur-Artal, J. M. M. Montiel, and J. D. Tardos, "ORB-SLAM: A Versatile and Accurate Monocular SLAM System," IEEE Transactions on Robotics, 2015. https://ieeexplore.ieee.org/document/7219438

18.4.2 Core Contributions

1. Builds a complete monocular SLAM system;
2. uses ORB features across the front end and loop detection;
3. supports real-time execution;
4. demonstrates strong performance on public datasets;
5. emphasizes keypoint selection and map point survival.

18.4.3 Relevance to This Project

ORB-SLAM shows that keypoint detection is not an isolated task but a critical part of the SLAM front end. This project should eventually move from corner maps to repeatability, matching score, and pose estimation impact.

18.4.4 Limitations

Directly inserting a QNN detector into ORB-SLAM is engineering-heavy. A short-term version should first evaluate repeatability, matching score, and RANSAC inlier ratio.

19 Detailed Review of Quantum Algorithms for Image Edge / Vision Tasks

19.1 Yao et al., "Quantum Image Processing and Its Application to Edge Detection: Theory and Experiment"

19.1.1 Positioning

This is a representative quantum image processing paper on edge detection. It is not QML and not a trainable QNN. It is a fixed quantum image processing algorithm that encodes an image into a quantum state and performs edge detection with a quantum circuit.[5]

19.1.2 Quantum Image Representation

The paper uses an amplitude-based representation. For an image with 2^n pixels:

$$|f\rangle = \sum_{i=0}^{2^n-1} c_i |i\rangle,$$

where $|i\rangle$ represents the pixel position and c_i is the normalized intensity.

The advantage is compact storage: n qubits can represent amplitudes over 2^n pixel positions. The major challenge is classical data loading: preparing a general image state can be expensive.

[5] X.-W. Yao et al., "Quantum Image Processing and Its Application to Edge Detection: Theory and Experiment," Physical Review X, 2017. https://arxiv.org/abs/1801.01465

19.1.3 Quantum Algorithm: QHED

The paper proposes **Quantum Hadamard Edge Detection** (QHED). The key observation is that a Hadamard gate can create sums and differences of neighboring amplitudes. For a local pair:

$$(c_{2i}, c_{2i+1}),$$

a Hadamard operation gives terms like:

$$\frac{c_{2i} + c_{2i+1}}{\sqrt{2}}, \quad \frac{c_{2i} - c_{2i+1}}{\sqrt{2}}.$$

The difference term corresponds to local image contrast, i.e. edge information.

The basic flow is:

1. encode the image as a quantum state;
2. apply a Hadamard gate to a position qubit;
3. extract neighboring-pixel difference information;
4. repeat or rearrange for different directions;
5. output an edge map.

19.1.4 Algorithmic Form

Abstractly:

$$|f\rangle = \sum_i c_i |i\rangle,$$

and after a Hadamard operation:

$$|g\rangle = (I \otimes H)|f\rangle.$$

Some output amplitudes correspond to local differences:

$$g_i \sim c_i - c_{i+1}.$$

19.1.5 Results

The paper gives numerical examples and experimental proof-of-principle demonstrations. It includes edge detection on 256×256 images in simulation and small experimental demonstrations on a quantum information processor. Reported output-state fidelities in the experiment were approximately between 0.972 and 0.981, with small Euclidean distances between experimental and theoretical output images.

19.1.6 Relevance to This Project

This paper suggests that:

1. edge detection can be written as a difference operation on quantum amplitudes;
2. corner detection can be seen as requiring strong differences along multiple directions;
3. Harris features such as $I_x, I_y, I_x^2, I_y^2, I_x I_y$ can be viewed as second-order statistics of such differences;
4. to move from edge detection to corner detection, one needs structure tensors or Hessian-like operators.

19.1.7 What Cannot Be Directly Reused

QHED outputs edge maps, not corner / keypoint probabilities. It has no descriptor, repeatability score, matching score, or SLAM front-end evaluation. Moreover, the theoretical efficiency usually does not include full classical image loading and complete readout costs.

19.2 Shubha et al., "Edge Detection Quantumized: A Novel Quantum Algorithm for Image Processing"

19.2.1 Positioning

This 2024 arXiv work improves on traditional QHED. The authors argue that the original QHED, often based on QPIE, is more suitable for binary images and may produce noisy or incomplete outlines on grayscale and complex images.[6]

19.2.2 Quantum Image Representation: FRQI

The paper uses **FRQI** (Flexible Representation of Quantum Images). A typical FRQI state is:

$$|I\rangle = \frac{1}{2^n} \sum_{i=0}^{2^{2n}-1} \left(\cos\theta_i |0\rangle + \sin\theta_i |1\rangle\right) |i\rangle.$$

Here $|i\rangle$ is the pixel position, and θ_i encodes grayscale intensity as a rotation angle on a color qubit.

19.2.3 Quantum Algorithm Form

The pipeline is roughly:

1. encode the classical image using FRQI;
2. partially measure the color qubit;
3. collapse the state into a QPIE-like amplitude representation;
4. reset or reuse the measured qubit;
5. apply modified QHED;
6. perform horizontal and vertical post-processing;

[6] Shubha et al., "Edge Detection Quantumized: A Novel Quantum Algorithm for Image Processing," arXiv:2404.06889, 2024. https://arxiv.org/abs/2404.06889

7. use dynamic thresholding and edge shifting to reduce noise and fix object outlines.

This is a hybrid pipeline:

FRQI encoding → partial measurement → QHED-like difference → classical threshold/post-processing.

19.2.4 Difference from Original QHED

The paper adds:

- FRQI encoding;
- partial measurement;
- dynamic thresholding;
- scanning-order-based edge shifting;
- classical post-processing.

19.2.5 Results

The paper primarily provides visual comparisons between traditional QHED and modified QHED. It reports that modified QHED produces better object outlines, works better on grayscale/complex images, and reduces noisy edges. However, it does not provide a comprehensive benchmark using metrics such as F1, IoU, PR-AUC, or BSDS500-style evaluation.

19.2.6 Relevance to This Project

The paper suggests:

1. FRQI + partial measurement can bridge image encoding and edge extraction;
2. dynamic thresholding can inspire adaptive thresholding after QNN corner probability output;
3. quantum edge detection still requires classical post-processing, so a hybrid pipeline is natural:

classical gradient features → QNN classifier → classical NMS / threshold → keypoint set.

19.2.7 What Cannot Be Directly Reused

The paper is still about edge detection, not corner detection. It produces edge outlines rather than stable keypoints. For SLAM/AR/robotics, keypoint repeatability and matchability matter more than visually pleasing edges.

19.3 Mogos et al., "Quantum Image Processing Using Edge Detection Based on Roberts Cross Operators"

19.3.1 Positioning

This work translates the classical Roberts Cross edge detector into a quantum image processing framework and tests it on IBM Quantum Experience and local simulators.[7]

19.3.2 Roberts Cross Operator

The classical Roberts Cross uses a 2×2 neighborhood to compute diagonal gradients:

$$G_x = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix}, \quad G_y = \begin{pmatrix} 0 & 1 \\ -1 & 0 \end{pmatrix}.$$

The edge magnitude is usually:

$$G = \sqrt{G_x^2 + G_y^2}$$

or approximately:

$$G = |G_x| + |G_y|.$$

19.3.3 Quantum Algorithm Form

This class of quantum Roberts methods usually follows:

1. encode the image using a quantum image representation;
2. implement differences between neighboring or diagonal pixels;
3. map the two Roberts masks into quantum operations;
4. recover an edge map through measurement and post-processing.

19.3.4 Experimental Results

The work reports tests on IBM Quantum Experience and local simulators. The results show that a quantum implementation of Roberts Cross can highlight regions with strong intensity changes.

19.3.5 Relevance to This Project

Corners require multiple gradient directions to be significant. Roberts Cross gives an example of diagonal gradient computation. Harris and Hessian features further organize multi-direction gradients into matrices:

[7] A. Mogos et al., "Quantum Image Processing Using Edge Detection Based on Roberts Cross Operators." https://scholar.xjtlu.edu.cn/en/publications/quantum-image-processing-using-edge-detection-based-onroberts-cro/

$$M = \sum_{W} \begin{pmatrix} I_x^2 & I_x I_y \\ I_x I_y & I_y^2 \end{pmatrix}.$$

Thus, this project can be viewed as moving from quantum edge operators to a quantum structure-tensor-based feature classifier.

19.3.6 What Cannot Be Directly Reused

Roberts Cross is sensitive to noise and detects edges, not corners. It is a fixed operator and does not learn from data.

19.4 Liu and Wang, "Quantum Image Edge Detection Based on Eight-direction Sobel Operator for NEQR"

19.4.1 Positioning

This work quantumizes Sobel edge detection and uses an eight-direction Sobel operator to reduce the loss of edge information compared with two-direction or four-direction Sobel variants.[8]

19.4.2 Quantum Image Representation: NEQR

NEQR stores grayscale values directly in computational basis states:

$$|I\rangle = \frac{1}{2^n} \sum_{y=0}^{2^n-1} \sum_{x=0}^{2^n-1} |f(y,x)\rangle |yx\rangle.$$

where $|yx\rangle$ represents pixel coordinates and $|f(y,x)\rangle$ stores the grayscale value.

NEQR is better suited for quantum comparison, subtraction, and thresholding, but requires more qubits and deeper circuits.

19.4.3 Quantum Algorithm Form

The algorithm:

1. encodes the image using NEQR;
2. computes Sobel gradients in multiple directions;
3. designs quantum circuits for gradient computation;
4. performs non-maximum suppression;
5. performs double thresholding;
6. performs edge tracking;
7. outputs an edge map.

[8] Liu and Wang, "Quantum Image Edge Detection Based on Eight-direction Sobel Operator for NEQR," 2022.

19.4.4 Complexity and Results

For a $2^n \times 2^n$ image with q-bit grayscale, the paper claims complexity:

$$O(n^2 + q^2).$$

Simulations show that eight-direction Sobel detects more edge information, especially diagonal edges, than two-direction or four-direction versions.

19.4.5 Relevance to This Project

Corner and junction detection depend on multi-directional structure. The eight-direction idea can inspire multi-direction local features:

$$x = [G_0, G_{\pi/4}, G_{\pi/2}, G_{3\pi/4}, G_{\pi}, ...].$$

These can be encoded into a QNN and combined through entangling layers to learn junction-like patterns.

19.4.6 What Cannot Be Directly Reused

The method still outputs an edge map. Its quantum arithmetic circuits may be too deep for a near-term small-scale experiment. For a short-term project, classical Sobel/Hessian features + small QNN is more feasible.

19.5 Henderson et al., "Quanvolutional Neural Networks: Powering Image Recognition with Quantum Circuits"

19.5.1 Positioning

Quanvolutional neural networks are an early attempt to use quantum circuits as image feature extractors. They are not QHED and not strict quantum speedup algorithms, but hybrid quantum-classical neural networks.[9]

19.5.2 Quantum Algorithm Form: Quanvolutional Layer

A quanvolutional layer resembles a classical convolution, but each local patch is processed by a small quantum circuit:

1. take a local patch, e.g. 2×2;
2. encode patch pixels into qubits;
3. apply a random or parameterized quantum circuit;
4. measure multiple qubits;
5. use measurement outputs as new feature channels;

[9] M. Henderson et al., "Quanvolutional Neural Networks: Powering Image Recognition with Quantum Circuits," Quantum Machine Intelligence, 2020. https://arxiv.org/abs/1904.04767

6. feed them into a classical classifier.

Formally:

$$P_{ij} \to U(P_{ij}) \to [\langle Z_1 \rangle, \langle Z_2 \rangle, ...] \to \text{classical classifier.}$$

19.5.3 Relation to Classical Convolution

Classical convolution is:

$$y_{ij} = \sum_{a,b} W_{ab} x_{i+a,j+b}.$$

Quanvolution is:

$$y_{ij}^{(k)} = \langle 0 | U^\dagger(P_{ij}) O_k U(P_{ij}) | 0 \rangle.$$

It can be seen as a nonlinear local feature map, possibly involving entanglement.

19.5.4 Results

The original paper compares classical CNNs, QNN/quanvolutional models, and classical models with extra nonlinearities on MNIST. It reports that the quanvolutional model can improve testing accuracy and training speed over its selected classical baseline. Such results depend on specific model size and training settings and should not be interpreted as general quantum advantage.

19.5.5 Relevance to This Project

This is one of the closest ideas to a patch-level vision QML task. A quanvolutional keypoint detector can be written as:

$$P_i \to \text{quantum local transform} \to p_{\text{corner}}.$$

The input can be raw pixels, gradient patches, or Hessian-like features. The output should be corner probability rather than image class.

19.5.6 What Cannot Be Directly Reused

The original task is image classification, not keypoint localization. It does not address localization error, repeatability, descriptor matching, SLAM tracking success, or NMS.

19.6 Hur et al., "Quantum Convolutional Neural Network for Classical Data Classification"

19.6.1 Positioning

This work benchmarks fully parameterized QCNNs for classical data classification. It is not an edge detector, but it is useful for QNN architecture design.[10]

19.6.2 Quantum Algorithm Form: QCNN

QCNN borrows the hierarchical reduction idea from classical CNNs. It contains:

1. quantum convolution layers;
2. quantum pooling layers.

A typical structure is:

encoding → quantum convolution → quantum pooling → quantum convolution → pooling → measurement.

Quantum convolution layers use two-qubit parameterized gates to capture local correlations. Quantum pooling reduces the number of qubits.

19.6.3 Data Encoding

The paper compares several encodings:

1. amplitude encoding;
2. qubit encoding;
3. dense angle encoding;
4. hybrid direct encoding;
5. hybrid angle encoding.

Amplitude encoding uses fewer qubits but may require deep state preparation. Angle-based methods are easier to implement but may require more qubits.

19.6.4 Results

The paper reports binary classification experiments on MNIST and Fashion-MNIST. Its 8-qubit QCNN uses few parameters, roughly between 12 and 51. Reported best accuracy is close to 99% for MNIST binary classification and around 94% for Fashion-MNIST binary classification. It also discusses nearest-neighbor QCNN connectivity, which is more suitable for NISQ hardware.

19.6.5 Relevance to This Project

This work can guide QNN design:

[10] T. Hur, L. Kim, and D. K. Park, "Quantum Convolutional Neural Network for Classical Data Classification," Quantum Machine Intelligence, 2022. https://arxiv.org/abs/2108.00661

1. use 4, 6, or 8 qubits;
2. use shallow parameterized two-qubit gates;
3. compare different encodings;
4. perform ansatz ablations;
5. report parameter count, depth, and two-qubit gate count.

For corner/keypoint detection, replace image-class embedding with patch feature embedding:

$$x = [I_x, I_y, I_{xx}, I_{xy}, I_{yy}, R, \lambda_1, \lambda_2].$$

19.6.6 What Cannot Be Directly Reused

MNIST/Fashion-MNIST classification is image-level classification, not dense or local keypoint detection. Corner/keypoint detection is highly imbalanced and cares about localization and repeatability.

19.7 Slabbert and Petruccione, "Hybrid Quantum-Classical Feature Extraction Approach for Image Classification Using Autoencoders and Quantum SVMs"

19.7.1 Positioning

This work is highly relevant methodologically. It explicitly argues that because NISQ devices have noise, scalability, readout, and gate-time constraints, one should not directly input high-dimensional images into a quantum model. Instead, classical feature extraction should reduce dimensionality first.[11]

19.7.2 Algorithm Form

The pipeline is:

Image → Classical convolutional autoencoder → Low-dimensional latent feature → QSVM / QOCSVM → Predictio

The classical autoencoder compresses the image. The quantum component uses a quantum feature map to construct a kernel for SVM classification.

19.7.3 QSVM Form

A quantum kernel is:

$$K(x_i, x_j) = |\langle \phi(x_i) | \phi(x_j) \rangle|^2,$$

where:

$$|\phi(x)\rangle = U_\phi(x)|0\rangle.$$

[11] J. Slabbert and F. Petruccione, "Hybrid Quantum-Classical Feature Extraction Approach for Image Classification Using Autoencoders and Quantum SVMs," 2024. https://arxiv.org/abs/2410.18814

The classifier is still an SVM:

$$f(x) = \text{sign}\left(\sum_i \alpha_i y_i K(x_i, x) + b\right).$$

19.7.4 Results

The paper uses datasets such as HTRU, MNIST, and CIFAR-10. It reports good reconstruction and classification performance on simpler datasets such as MNIST, but significantly weaker performance on more complex images such as CIFAR-10. It emphasizes the balance between classical feature extraction and quantum prediction.

19.7.5 Relevance to This Project

This paper directly supports the design principle:

> do not directly input the full image into a quantum model; first extract low-dimensional meaningful features, then use a quantum classifier.

For this project, replace the autoencoder with classical visual features:

$$P_i \to [I_x, I_y, I_{xx}, I_{xy}, I_{yy}, R, \lambda_1, \lambda_2] \to \text{QNN/QSVM} \to p_{\text{corner}}.$$

Two quantum variants can be tested:

1. VQC/QNN classifier;
2. QSVM classifier.

19.7.6 What Cannot Be Directly Reused

The paper is about image classification, not keypoint detection. Its learned features are global image features rather than local geometric features.

19.8 Senokosov et al., "Quantum Machine Learning for Image Classification"

19.8.1 Positioning

This work proposes hybrid quantum-classical image classification models, including parallel quantum circuits and quanvolutional layers. It is still image classification, but it is useful for integrating quantum layers into classical networks.[12]

19.8.2 Quantum Algorithm Form

One model uses multiple parallel variational quantum circuits. Each circuit processes part of the classical features and outputs measurements as intermediate features.

[12] A. Senokosov et al., "Quantum Machine Learning for Image Classification," Machine Learning: Science and Technology, 2024. https://arxiv.org/abs/2304.09224

Another model uses a quanvolutional layer:

$$\text{local image patch} \rightarrow \text{quantum circuit} \rightarrow \text{measurement features} \rightarrow \text{classical layers.}$$

19.8.3 Results

The paper reports around 99.21% accuracy on full MNIST, over 99% on Medical MNIST, and over 82% on CIFAR-10. The quanvolutional model is reported to be close to its classical counterpart while using fewer trainable parameters, and to outperform a classical model with the same parameter count.

19.8.4 Relevance to This Project

This paper suggests two possible routes:

1. **Quantum local patch transform**

$$P_i \rightarrow \text{quanvolution} \rightarrow \text{MLP} \rightarrow p_{\text{corner}}.$$

2. **Classical feature + quantum classifier**

$$\text{gradient/Hessian features} \rightarrow \text{VQC} \rightarrow p_{\text{corner}}.$$

19.8.5 What Cannot Be Directly Reused

The evaluation is image-level accuracy, not keypoint repeatability or localization error. For SLAM/AR/robotics, high classification accuracy does not necessarily mean useful keypoints.

20 How to Transfer Quantum Algorithm Routes to Corner / Keypoint Detection

The quantum-related literature can be grouped into three technical routes:

Route	Representative algorithms	Typical task	Suitability for this project
Quantum image processing	QHED, quantum Roberts, quantum Sobel	Edge map	Useful as edge/gradient operator references
Quanvolution / QCNN	Quanvolutional layer, QCNN	Image classification	Can be adapted to patch classification
Hybrid feature + quantum classifier	Autoencoder + QSVM, VQC	Image classification	Most suitable for NISQ-stage transfer

The recommended route is not full quantum image processing, but:

classical local feature extraction → QNN / QSVM → corner probability → NMS → keypoint set.

More concretely:

$$P_i \to x_i = [I_x, I_y, I_{xx}, I_{xy}, I_{yy}, I_x^2, I_y^2, I_x I_y, \lambda_1, \lambda_2, R] \to f_\theta(x_i) \to p_i.$$

where f_θ may be:

1. a variational quantum classifier;
2. a small QCNN;
3. a QSVM;
4. a quanvolution + classical classifier.

21 Suggested Quantum Model Experiments

21.1 VQC Version

Input:

$$x_i \in \mathbb{R}^d.$$

Encoding:

$$U_{\text{enc}}(x) = \prod_j R_y(\alpha x_j) R_z(\beta x_j).$$

Variational layer:

$$U(\theta) = \prod_{\ell=1}^{L} \left[\prod_j R_y(\theta_{\ell j}^{(1)}) R_z(\theta_{\ell j}^{(2)}) \cdot \prod_j \text{CNOT}_{j,j+1} \right].$$

Output:

$$p_{\text{corner}} = \sigma\left(\langle Z_0 \rangle\right).$$

Ablations:

- no entanglement;
- ring entanglement;
- full entanglement;
- data re-uploading;

- different depths L.

21.2 QCNN Version

The circuit structure is:

$$\text{encoding} \to \text{two-qubit convolution} \to \text{pooling} \to \text{two-qubit convolution} \to \text{measurement}.$$

Compare:

- 4-qubit QCNN;
- 6-qubit QCNN;
- 8-qubit QCNN;
- nearest-neighbor QCNN;
- fully connected QCNN.

Output:

$$p_{\text{corner}} = \sigma(\langle Z \rangle).$$

21.3 QSVM Version

Quantum feature map:

$$|\phi(x)\rangle = U_\phi(x)|0\rangle.$$

Quantum kernel:

$$K(x_i, x_j) = |\langle \phi(x_i) | \phi(x_j) \rangle|^2.$$

Classifier:

$$f(x) = \text{sign}\left(\sum_i \alpha_i y_i K(x_i, x) + b \right).$$

This is suitable for small-data, low-dimensional patch classification.

21.4 Quanvolution Version

For each patch:

$$P_i \to U(P_i) \to [\langle Z_1 \rangle, \dots, \langle Z_m \rangle] \to \text{MLP} \to p_{\text{corner}}.$$

This version is closest to a classical CNN, but the output is whether the center point is a keypoint rather than an image class.

22 Difference from Existing Work

Existing work roughly falls into three categories:

1. **Classical corner/keypoint detection**: Harris, FAST, ORB;
2. **Classical SLAM front ends**: ORB-SLAM and related systems;
3. **Quantum image processing / QML image classification**: QHED, FRQI edge detection, QCNN, hybrid quantum-classical image classification.

There is still no mature standard line of work on:

> QML / QNN for corner, junction, and stable keypoint extraction.

Thus, the innovation is not to build another quantum edge detector, but to:

> move from edge detection to corner/keypoint detection and connect it with stable local features needed in SLAM, AR, and robotics.

23 Recommended Short-term Version

The first version should be:

> **Quantum local feature extractor for noisy corner and keypoint detection**

Concrete contributions:

1. use patch-level QNN instead of full-image quantum encoding;
2. use gradients + structure tensor + Hessian-like features instead of raw pixels;
3. output corner/keypoint probability instead of edge map;
4. compare with Harris, FAST, ORB, and small MLP;
5. conduct entanglement ablation;
6. test robustness under noise, low resolution, blur, and illumination change;
7. report qubits, depth, two-qubit gates, parameter count, and inference cost.

24 Recommended Experimental Route

24.1 Stage 1: Synthetic Data

- generate L/T/X/Y junctions, checkerboards, polygon corners;
- generate ground-truth corner labels;
- add noise, blur, and illumination changes;
- train QNN to classify whether a patch center is a corner;
- compare with Harris, FAST, and MLP.

24.2 Stage 2: Real-image Pseudo-labels

- use Harris / FAST / ORB to generate pseudo-labels;
- train QNN to learn keypoint probability;
- test repeatability under noise and transformations.

24.3 Stage 3: Matching-level Evaluation

- detect keypoints in image pairs;
- use ORB / BRIEF descriptors;
- compute matching score and RANSAC inlier ratio;
- check whether QNN detection improves stable matching.

24.4 Stage 4: SLAM Front-end Validation

- optionally integrate into ORB-SLAM or a lightweight visual odometry pipeline;
- compare tracking success and pose error;
- this is engineering-heavy and should be an extension goal.

25 Final Judgment on the Quantum Algorithm Literature

The mature literature is mostly about quantum edge detection and quantum image classification, not quantum corner/keypoint extraction. In other words:

> existing work shows that quantum algorithms can process edge differences, local image transformations, and small-scale image classification, but has not yet established a standard QML keypoint detector for SLAM/AR/robotics.

Therefore, a reasonable contribution should be:

1. move from quantum edge detection to quantum keypoint detection;
2. use patch-level low-dimensional feature encoding instead of full-image quantum encoding;
3. replace image-level accuracy with repeatability, localization error, and matching score;
4. frame the goal as low-parameter QNN robustness rather than quantum advantage;
5. compare VQC, QCNN, QSVM, and quanvolution routes systematically.

The recommended main model is:

Gradient / Structure Tensor / Hessian Features → Low-depth VQC or QCNN → Corner Probability → NMS → St

26 One-sentence Summary

The core of this project is not "a quantum model sees images," but:

> reformulating local corner and keypoint detection used in SLAM/AR/robotics as a low-resource, patch-level, ablation-friendly QML feature classification task; using Harris, FAST,

and ORB as strong baselines; and focusing on noise robustness, repeatability, and circuit resource cost.

27 References and Resources

- Harris, C. and Stephens, M. "A Combined Corner and Edge Detector." Proceedings of the 4th Alvey Vision Conference, 1988. https://www.bmva-archive.org.uk/bmvc/1988/avc-88-023.pdf
- Rosten, E. and Drummond, T. "Machine Learning for High-Speed Corner Detection." ECCV 2006. https://www.edwardrosten.com/work/rosten_2006_machine.pdf
- Rublee, E., Rabaud, V., Konolige, K., and Bradski, G. "ORB: An efficient alternative to SIFT or SURF." ICCV 2011. https://ieeexplore.ieee.org/document/6126544
- Mur-Artal, R., Montiel, J. M. M., and Tardos, J. D. "ORB-SLAM: A Versatile and Accurate Monocular SLAM System." IEEE Transactions on Robotics, 2015. https://ieeexplore.ieee.org/document/7219438
- Yao, X.-W. et al. "Quantum Image Processing and Its Application to Edge Detection: Theory and Experiment." Physical Review X, 2017. https://arxiv.org/abs/1801.01465
- Shubha et al. "Edge Detection Quantumized: A Novel Quantum Algorithm for Image Processing." arXiv:2404.06889, 2024. https://arxiv.org/abs/2404.06889
- Mogos, A. et al. "Quantum Image Processing Using Edge Detection Based on Roberts Cross Operators." https://scholar.xjtlu.edu.cn/en/publications/quantum-image-processing-using-edge-detection-based-onroberts-cro/
- Henderson, M. et al. "Quanvolutional Neural Networks: Powering Image Recognition with Quantum Circuits." Quantum Machine Intelligence, 2020. https://arxiv.org/abs/1904.04767
- Hur, T., Kim, L., and Park, D. K. "Quantum Convolutional Neural Network for Classical Data Classification." Quantum Machine Intelligence, 2022. https://arxiv.org/abs/2108.00661
- Slabbert, J. and Petruccione, F. "Hybrid Quantum-Classical Feature Extraction Approach for Image Classification Using Autoencoders and Quantum SVMs." 2024. https://arxiv.org/abs/2410.18814
- Senokosov, A. et al. "Quantum Machine Learning for Image Classification." Machine Learning: Science and Technology, 2024. https://arxiv.org/abs/2304.09224